Generic Classes in C#.NET

1. What are Generics?

A **Generic Class** allows you to create **classes, methods, interfaces, and collections that work with any data type** while maintaining **type safety**.

Instead of writing separate classes for each data type, you write **one reusable class**.

Without Generics

Imagine we create a class to store integer values.

```
class IntBox
{
    private int value;

    public void SetValue(int v)
    {
        value = v;
    }

    public int GetValue()
    {
        return value;
    }
}
```

Now if we want to store **string**:

```
class StringBox
{
    private string value;
```

```
    public void SetValue(string v)
    {
        value = v;
    }

    public string GetValue()
    {
        return value;
    }
}
```

Problem:

❌ Duplicate code
❌ Poor reusability
❌ Difficult maintenance

---

2. Solution — Generic Classes

Generics allow us to define a **type parameter**.

Syntax

```
class ClassName<T>
{
}
```

T represents a **Type Parameter**.

Practical Demo 1: Basic Generic Class

Step 1: Create Generic Class

```
class Box<T>
{
    private T value;

    public void SetValue(T v)
    {
        value = v;
    }

    public T GetValue()
    {
        return value;
    }
}
```

Step 2: Use the Generic Class

```
class Program
{
    static void Main()
    {
        Box<int> intBox = new Box<int>();
        intBox.SetValue(100);

        Console.WriteLine(intBox.GetValue());

        Box<string> strBox = new Box<string>();
        strBox.SetValue("Hello Generics");
```

```
        Console.WriteLine(strBox.GetValue());
    }
}
```

Output

100

Hello Generics

Explanation

Box<int> → T becomes int

Box<string> → T becomes string

The **same class works for multiple data types**.

Practical Demo 2: Generic Class with Multiple Types

Generic classes can have **multiple type parameters**.

Example: Key Value Pair

```
class KeyValue<T1, T2>
{
    public T1 Key { get; set; }
    public T2 Value { get; set; }

    public void Display()
    {
        Console.WriteLine($"Key: {Key} Value: {Value}");
    }
}
```

Usage

```
class Program
{
    static void Main()
    {
        KeyValue<int, string> obj = new KeyValue<int, string>();

        obj.Key = 101;
        obj.Value = "Laptop";

        obj.Display();
    }
}
```

Output

Key: 101 Value: Laptop

Practical Demo 3: Generic Class with Methods

```
class Calculator<T>
{
    public void Display(T value)
    {
        Console.WriteLine("Value is: " + value);
    }
}
```

Usage

```
class Program
{
```

```
    static void Main()
    {
        Calculator<int> obj1 = new Calculator<int>();
        obj1.Display(500);

        Calculator<string> obj2 = new Calculator<string>();
        obj2.Display("Welcome");
    }
}
```

Practical Demo 5: Generic Class in Real World (Repository Pattern)

Generics are heavily used in **ASP.NET Core**.

Example: Generic Repository.

```
class Repository<T>
{
    public void Add(T item)
    {
        Console.WriteLine($"{item} added to database");
    }

    public void Delete(T item)
    {
        Console.WriteLine($"{item} deleted from database");
    }
}
```

Usage

```
class Program
{
    static void Main()
    {
        Repository<string> repo1 = new Repository<string>();
        repo1.Add("Product1");

        Repository<int> repo2 = new Repository<int>();
        repo2.Add(101);
    }
}
```

---

3. Advantages of Generics

1️⃣ **Code Reusability**

Write once — use for many types.

2️⃣ **Type Safety**

Compile-time checking.

Example:

```
Box<int> box = new Box<int>();
box.SetValue("Hello");  // Compile error
```

3️⃣ **Performance**

Generics avoid **boxing and unboxing**.

Non-generic collections:

ArrayList

Generic collections:

List<T>

Dictionary<TKey,TValue>

Queue<T>

Stack<T>

---

4. Generic Naming Conventions

Name	Meaning
T	Type
TKey	Key type
TValue	Value type
TResult	Result type
TItem	Item type

Example:

Dictionary<TKey, TValue>

---

5. Generic Collections in .NET

Common generic collections:

List<T>

Dictionary<TKey,TValue>

Queue<T>

Stack<T>

HashSet<T>

---

6. Generic Constraints

Constraints restrict the types used in generics.

Example:

```
class Test<T> where T : class
{
}
```

Types of constraints:

Constraint	Meaning
where T : class	reference type
where T : struct	value type
where T : new()	must have constructor
where T : BaseClass	inherit base class
where T : interface	implement interface

Example:

```
class Factory<T> where T : new()
{
    public T Create()
    {
        return new T();
    }
}
```

---

7. Generic vs Object Type

Object Type

```
class Box
{
    object value;
}
```

Problems:

❌ Runtime errors
❌ Boxing/Unboxing
❌ No type safety

Generic Type

```
class Box<T>
{
    T value;
}
```

Advantages:

✓ Compile-time safety
✓ Better performance
✓ Reusable code

---

8. Real-World Use in ASP.NET Core

You will often see generics in:

IRepository<T>

ILogger<T>

DbSet<T>

List<T>

Task<T>